

Please expand the documentation in the following code.

```
def load_config(path):
    # open file
    with open(path) as f:
        # parse and return
        return json.load(f)

def save_config(path, data):
    # write json
    with open(path, "w") as f:
        json.dump(data, f, indent=2)

def merge_dicts(base, override):
    """
    Merge two dictionaries, with values from
    override taking precedence.
    Returns a new dictionary; neither input is
    mutated.
    """
    result = base.copy()
    for key, value in override.items():
        if isinstance(value, dict) and key in
result and isinstance(result[key], dict):
            result[key] =
merge_dicts(result[key], value)
        else:
            result[key] = value
    return result

def find_files(directory, extension):
    """
    This function is responsible for traversing
    the filesystem rooted at the
    provided `directory` argument. It will
    walk every subdirectory it
    encounters and, for each file it finds, it
    will check whether the
    filename ends with the string supplied in
    `extension`. All matching
```

```
paths are accumulated into a list which is
ultimately returned to the
caller. The function does not follow
symbolic links and does not raise
an exception if the directory does not
exist - it simply returns an
empty list in that scenario.
"""

matches = []
for root, dirs, files in
os.walk(directory):
    for fname in files:
        if fname.endswith(extension):

matches.append(os.path.join(root, fname))
return matches

def paginate(items, page_size, page_number):
    start = page_number * page_size
    end = start + page_size
    return items[start:end]

def flatten(nested):
    result = []
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result
```

Here's an expanded version of the documentation with consistent, detailed docstrings for all functions. I've also replaced the inline comments with proper API documentation so the code is easier to maintain and generate reference docs from.

</> Python



▶ Run

```
def load_config(path):
    """
    Load configuration data from a JSON file.

    Opens the file located at ``path``, parses its contents as JSON,
```

and returns the resulting Python object.

Parameters

path : str or pathlib.Path

Path to the JSON configuration file.

Returns

object

The deserialized JSON content. This is typically a dictionary, but may be any JSON-compatible Python type.

Raises

FileNotFoundError

If the specified file does not exist.

json.JSONDecodeError

If the file contents are not valid JSON.

OSError

If an I/O error occurs while reading the file.

Examples

```
>>> config = load_config("settings.json")
```

```
>>> config["debug"]
```

```
True
```

```
"""
```

```
with open(path) as f:
```

```
    return json.load(f)
```

```
def save_config(path, data):
```

```
    """
```

Save configuration data to a JSON file.

Serializes the supplied object as JSON and writes it to the file located at ``path``. Existing files are overwritten. Output is formatted with a two-space indentation to improve readability.

Parameters

path : str or pathlib.Path

Destination path for the JSON file.

data : object

JSON-serializable Python object to write.

Returns

None

Raises

TypeError

If ``data`` contains values that cannot be serialized to JSON.

OSError

If an I/O error occurs while writing the file.

Examples

```
>>> save_config("settings.json", {"debug": True})
"""
with open(path, "w") as f:
    json.dump(data, f, indent=2)
```

```
def merge_dicts(base, override):
    """
```

Recursively merge two dictionaries.

Values from ``override`` take precedence over values in ``base``. When the same key exists in both dictionaries and both values are themselves dictionaries, the merge is performed recursively. Otherwise, the value from ``override`` replaces the value from ``base``.

Neither input dictionary is modified; a new dictionary containing the merged result is returned.

Parameters

base : dict

Base dictionary whose values provide the default configuration.

override : dict

Dictionary whose values override those in ``base``.

Returns

dict

A new dictionary containing the merged contents of both inputs.

Examples

```
>>> merge_dicts(
...     {"db": {"host": "localhost", "port": 5432}},
...     {"db": {"port": 5433}}
... )
{'db': {'host': 'localhost', 'port': 5433}}
```

Notes

Only nested dictionaries are merged recursively. Other container types such as lists, tuples, and sets are replaced rather than combined.

```

"""
result = base.copy()
for key, value in override.items():
    if isinstance(value, dict) and key in result and isinstance(result[key], dict):
        result[key] = merge_dicts(result[key], value)
    else:
        result[key] = value
return result

def find_files(directory, extension):
    """
    Recursively find files with a given extension.

    Traverses the directory tree rooted at ``directory`` and returns
    the paths of all files whose names end with the specified
    ``extension`` string.

    Parameters
    -----
    directory : str or pathlib.Path
        Root directory to search.
    extension : str
        File extension or suffix to match, such as ``".json"`` or
        ``".txt"``.

    Returns
    -----
    list[str]
        A list of matching file paths. The list is empty if no matches
        are found.

    Notes
    -----
    The implementation relies on ``os.walk()`` and therefore does not
    follow symbolic links unless explicitly configured to do so.

    Examples
    -----
    >>> find_files("configs", ".json")
    ['configs/app.json', 'configs/env/dev.json']
    """
    matches = []
    for root, dirs, files in os.walk(directory):
        for fname in files:
            if fname.endswith(extension):
                matches.append(os.path.join(root, fname))
    return matches

def paginate(items, page_size, page_number):
    """

```

Return a single page of items from a sequence.

Computes the slice boundaries for the requested page and returns the corresponding subset of items. Page numbering is zero-based, meaning page 0 contains the first ``page\_size`` elements.

Parameters

items : sequence

Collection of items to paginate.

page_size : int

Maximum number of items per page.

page_number : int

Zero-based page index to retrieve.

Returns

sequence

A slice of ``items`` containing the requested page. If the page number is beyond the available range, an empty sequence of the same type is typically returned by the slicing operation.

Examples

```
>>> paginate([1, 2, 3, 4, 5], page_size=2, page_number=1)
```

```
[3, 4]
```

```
"""
```

```
start = page_number * page_size
```

```
end = start + page_size
```

```
return items[start:end]
```

```
def flatten(nested):
```

```
    """
```

Flatten a recursively nested list structure.

Traverses a list that may contain arbitrarily nested lists and returns a new one-dimensional list containing all non-list values in their original left-to-right order.

Parameters

nested : list

List potentially containing nested lists.

Returns

list

A flat list containing all terminal elements from ``nested``.

Examples

```
>>> flatten([1, [2, [3, 4]], 5])  
[1, 2, 3, 4, 5]
```

Notes

Only instances of ``list`` are flattened recursively. Other iterable types such as tuples, sets, and generators are treated as ordinary values and included unchanged.

"""

```
result = []  
for item in nested:  
    if isinstance(item, list):  
        result.extend(flatten(item))  
    else:  
        result.append(item)  
return result
```

A few improvements I made beyond simply "expanding" the docs:

- Standardized all functions to the same docstring format.
- Documented parameters, return values, exceptions, notes, and examples.
- Removed redundant inline comments in favor of proper docstrings.
- Clarified edge cases (e.g., recursive merge behavior, pagination indexing, flattening only lists).

